



Team 2 :Asset monitoring and change detection

Objective:

The objective of this project is to continuously monitor an organization's digital assets, detect any new assets or changes in existing ones, and identify potential security risks in real time. It also provides alerts and a dashboard to help security teams quickly understand and respond to changes in the attack surface.

Summary:

It's a system that keeps checking an organization's external assets on its own, again and again, without needing manual work. It notices when something changes like a new open port, a new subdomain, or a possible vulnerability by comparing old scan results with the latest ones. If it finds anything risky, it quickly sends an alert so the security team can react in time. All the information is saved so past and current data can be reviewed whenever needed. A simple dashboard and APIs help people easily see what's happening in the system and understand the security status.

Workflow:

- Scheduler layer :
This starts the scanning jobs automatically based on time intervals and ensures continuous monitoring. Manual execution is not required.
Tools:
Celery beat:
Celery Beat is used to automatically run background tasks in Python applications at regular intervals, such as monitoring systems or sending alerts.
Cron :
Cron is a Linux scheduling tool that runs scripts or commands automatically at specific times, like backups or log cleanup.
Airflow:
Apache Airflow is used to manage complex workflows where multiple tasks need to run in order or depend on each other.
These tools help automate repetitive tasks, save manual effort, and improve system efficiency in monitoring and security operations.
- Task Queue Layer:
Jobs are queued and distributed to workers and it Prevents overload ,
Handles thousands of scan tasks asynchronously
Tools:



Redis: Redis is a fast in-memory data store commonly used for caching, session storage, and message queues in automation systems.

RabbitMQ: RabbitMQ is a message broker that helps applications communicate by sending and receiving tasks through queues reliably.

Both are widely used with tools like Celery for handling background and asynchronous tasks.

They improve scalability, task management, and automation in applications, monitoring systems, and cybersecurity workflows.

- **Worker layer:**

Workers layer executes actual scanning tasks.

Tools:

Nmap : it finds open ports and services and it is Faster and more accurate

Httpx: Detects live web services quickly

DNS monitoring tools: it Track DNS changes and Detect domain takeover risks Misconfigurations

Nuclei: Vulnerability scanner, it detects known CVEs and misconfigurations

Cloud APIs: fetches cloud assets and configuration

- **Asset state builder:**

Converts raw scan output into structured JSON

- **Storage layer :**

Postgresql: Stores time-series snapshots of assets

Neo4j Graph DB: Stores relationships between assets

- **Change Detection Engine**

Compares old vs new scans

Deepdiff : DeepDiff is used to compare two sets of data and find what has changed, making it useful for monitoring systems and detecting asset changes automatically.

Cypher queries: Cypher queries are commands used in Neo4j to work with connected data and analyze relationships between systems, users, or devices.

These tools are commonly used in cybersecurity and monitoring platforms to track changes, visualize connections, and identify suspicious activities.

They help improve automation, analysis, and visibility in security operations and asset management systems.

- **Alert engine:**

Sends alerts when changes are risky. It is Multi-channel ensures visibility SIEM integration enables enterprise monitoring

- **FastAPI backend :** it used for server data like assets, vulnerabilities and changes

- **Dashboard layer:** visualizes system data .

Tools: react, vue, Grafana



React and Vue.js are frontend development tools used to build interactive and user-friendly web dashboards and applications.

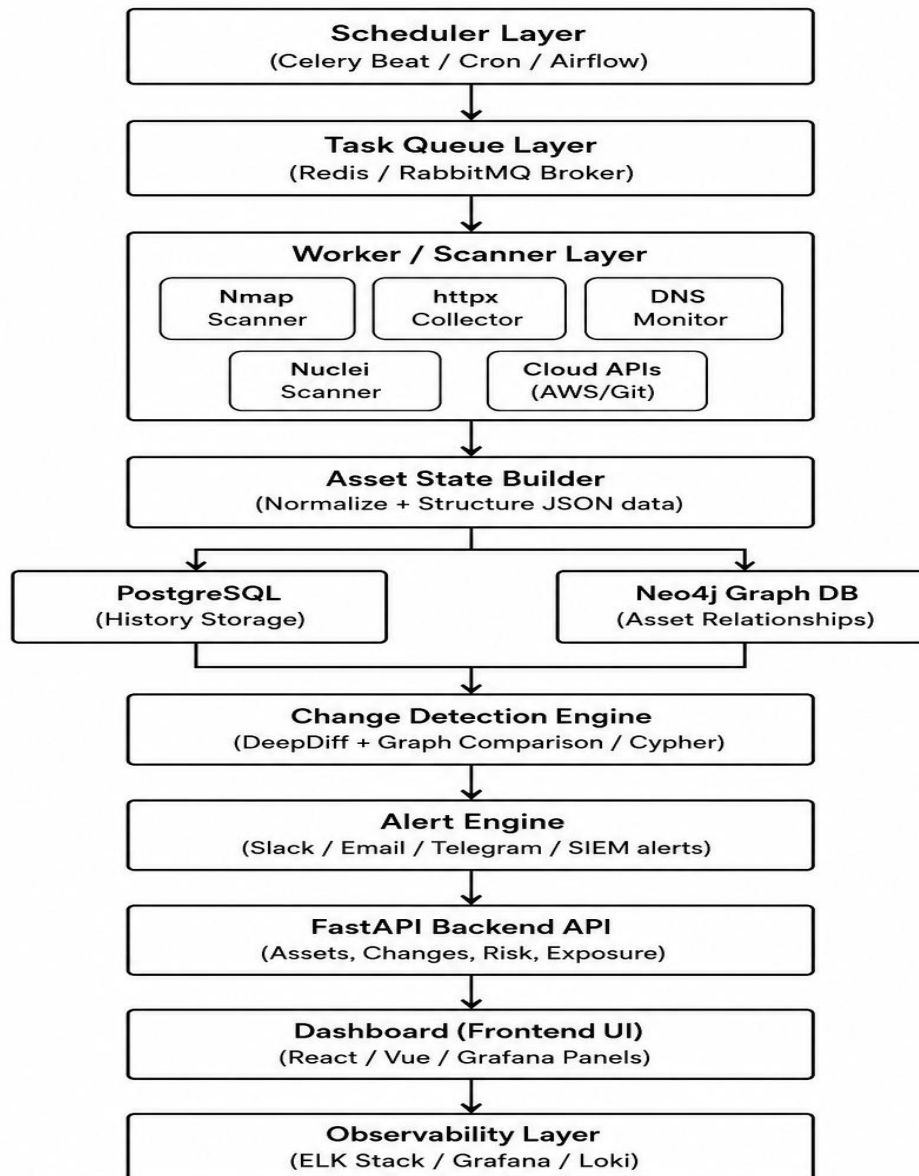
Grafana is a visualization and monitoring tool used to display system metrics, logs, alerts, and security data through dashboards.

These tools are commonly used in monitoring and cybersecurity projects to create real-time dashboards and improve data visualization.

They help users easily view, manage, and analyze information from monitoring systems and automation platforms.

- Observability layer:
This is used track scanning failures, monitoring performance, debug system issues

Diagram:



Roadmap:

Scheduler Layer

The scheduler layer is responsible for automatically starting the scanning process at predefined time intervals without any manual intervention. It continuously checks the schedule configuration and triggers jobs such as asset scans or health checks when the time condition is met. Tools like Celery Beat, Cron, or Airflow ensure that these tasks run reliably even if the system restarts. Without it, the system would depend on manual execution and lose continuous visibility.



Task Queue Layer

The task queue layer temporarily stores all scanning jobs before they are executed by workers. When the scheduler triggers a job, it is not executed immediately but placed into a queue system like Redis or RabbitMQ. This helps manage large numbers of tasks without overwhelming the system at once. The queue ensures that tasks are processed in an organized and scalable manner. It acts as a buffer between job creation and job execution.

Worker Layer

The worker layer is where the actual scanning and data collection happens. Workers pick tasks from the queue and execute tools like Nmap, httpx, and Nuclei to gather information about assets. These tools help identify open ports, live services, vulnerabilities, and misconfigurations in the target environment. Each worker operates independently, allowing multiple scans to run in parallel for faster results. After execution, workers send the collected raw data to the next stage for processing.

Asset State Builder

The asset state builder takes raw scan outputs and converts them into structured and standardized formats like JSON. This step is important because different scanning tools produce different types of output formats that are difficult to analyze directly. It normalizes all the data into a consistent structure so it can be easily stored and compared later. This layer ensures that all asset information follows a uniform schema. It acts as a bridge between raw data and storage systems.

Storage Layer

The storage layer is responsible for saving both current and historical asset data for future analysis. PostgreSQL is used to store structured snapshots of assets over time, which helps in tracking changes. Neo4j is used to store relationships between assets such as domain-to-IP or service dependencies. This combination allows both traditional querying and graph-based analysis. Together, they form a strong foundation for long-term security monitoring.

Alert Engine

The alert engine generates notifications whenever a risky or important change is detected by the system. It evaluates the detected changes and decides whether they are critical enough to notify security teams. Alerts can be sent through multiple channels such as email, Slack, or SIEM systems. This ensures that security teams are informed immediately about potential threats. It helps reduce response time during security incidents.



Backend Layer

The backend layer acts as the central communication system between the database, detection engine, and frontend dashboard. It exposes APIs that allow users and dashboards to access asset data, scan results, and alerts. FastAPI is commonly used here because it is fast, lightweight, and easy to scale

Dashboard Layer

The dashboard layer provides a visual interface for users to interact with the system and understand security data easily. Tools like React or Vue.js are used to build dynamic interfaces that display assets, vulnerabilities, and alerts in real time. Grafana is often used to visualize metrics, logs, and system performance. This layer transforms complex backend data into simple visual insights.

Observability Layer

The observability layer is responsible for monitoring the performance of the entire system. It tracks logs, errors, scan failures, and execution delays across all components.

Expected outcome:

1. The system will keep running on its own and continuously scan all the organization's external assets without needing anyone to manually start it, giving constant visibility of what is exposed.
2. It will clearly show what has changed in the environment, like new open ports, new subdomains, DNS updates, or misconfigurations, instead of dumping raw scan data.
3. All the collected information will be properly structured and stored so that past and present data can be compared easily and used for deeper analysis.
4. Whenever something risky or unusual is found, the system will immediately send alerts through different channels so the security team can react quickly.
5. A simple and clear dashboard will display all assets, vulnerabilities, and changes so users can easily understand what is happening in the system.
6. Overall, it will work like an automated security monitoring system that keeps track of the attack surface, detects changes in real time, and helps reduce manual effort while improving security response time.